



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Ασφάλεια Ασύρματων και Κινητών Δικτύων

Android-Use Case 2

Επιβλέποντες Καθηγητές: Χρήστος Ξενάκης και Γεώργιος Ευθύμογλου
(Γρηγόριος Παπουτσής)

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς
18/06/2026

Περίληψη

Η παρούσα εργασία διερευνά τις ευπάθειες μηχανισμών αυθεντικοποίησης σε εφαρμογές Android, αναλύοντας τρεις διαφορετικές μεθοδολογίες παράκαμψης (login bypass). Μέσω στατικής ανάλυσης με JADX και APKTool (Smali Patching) και αντίστροφης μηχανικής native βιβλιοθηκών με Ghidra, αποδεικνύεται ότι η απόκρυψη κωδικών δεν αποτελεί επαρκή ασφάλεια. Επιπλέον, μέσω δυναμικής ανάλυσης με το πλαίσιο Frida, επιτυγχάνεται παρέμβαση στη μνήμη κατά τον χρόνο εκτέλεσης (runtime), παρακάμπτοντας κάθε τοπικό έλεγχο. Τα αποτελέσματα επιβεβαιώνουν ότι οι client-side προστασίες είναι επισφαλείς, αναδεικνύοντας την ανάγκη για ισχυρή κρυπτογράφηση και server-side επαλήθευση.

Περιεχόμενα

Εισαγωγή.....	1
Πρακτικό κομμάτι εργασίας	2
Βιβλιογραφία	24

Εισαγωγή

Η ραγδαία εξέλιξη των λειτουργικών συστημάτων για φορητές συσκευές, με κυρίαρχο το Android, έχει μετατρέψει τα smartphones σε βασικά εργαλεία για την πρόσβαση σε ευαίσθητα προσωπικά και εταιρικά δεδομένα. Παράλληλα, η πολυπλοκότητα των εφαρμογών έχει δημιουργήσει μια νέα πρόκληση στον τομέα της κυβερνοασφάλειας, την προστασία του κώδικα και των μηχανισμών αυθεντικοποίησης που εκτελούνται στην πλευρά του χρήστη (client-side).

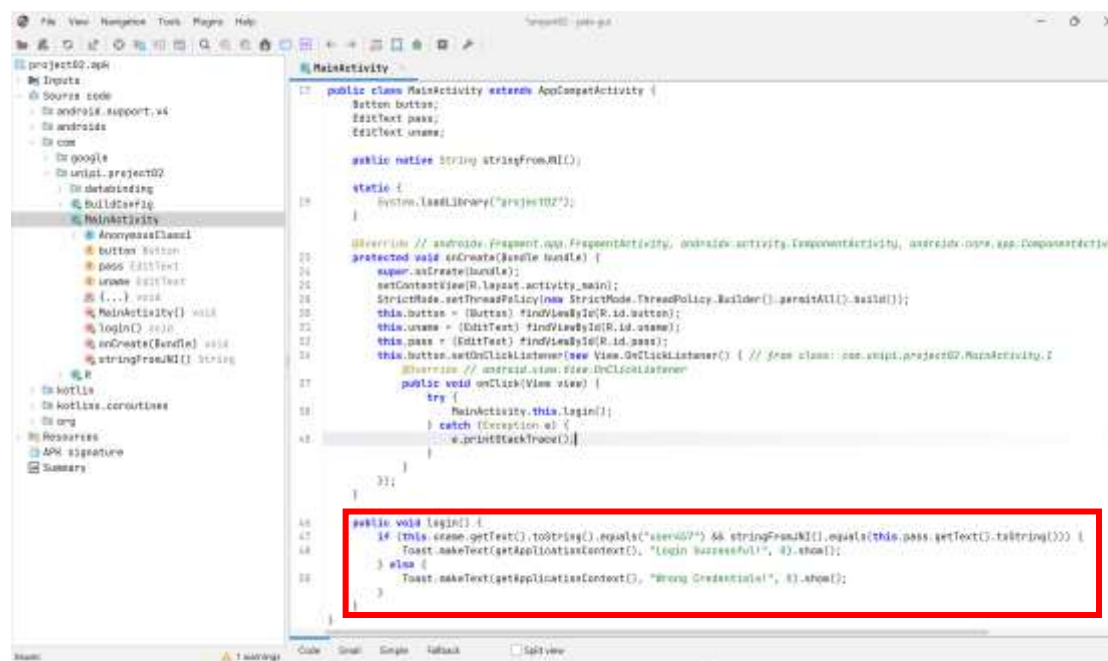
Η παρούσα εργασία επικεντρώνεται στην ανάλυση και την πρακτική αξιολόγηση των μηχανισμών ασφαλείας μιας εφαρμογής Android, η οποία επιχειρεί να θωρακίσει τον ευαίσθητο κώδικά της (Flag) μέσω της μεταφοράς της λογικής της σε εξωτερικές native βιβλιοθήκες (JNI-Java Native Interface). Στόχος της έρευνας δεν είναι μόνο η παραβίαση του συγκεκριμένου μηχανισμού, αλλά η διερεύνηση της αποτελεσματικότητας διαφορετικών μεθοδολογιών reverse engineering.

Κατά τη διάρκεια της ανάλυσης, εξετάζονται τρεις διακριτοί πυλώνες προσέγγισης:

1. **Στατική ανάλυση:** Μέσω της ανακατασκευής και τροποποίησης του κώδικα Smali, αναδεικνύεται η ευπάθεια των στατικών ελέγχων.
2. **Αντίστροφη μηχανική native κώδικα:** Με τη χρήση εργαλείων απομεταγλώττισης, αναλύεται η δομή της βιβλιοθήκης C/C++ για τον εντοπισμό κρυφών διαδρομών.
3. **Δυναμική ανάλυση:** Μέσω του πλαισίου Frida, επιτυγχάνεται η χειραγώγηση της μνήμης της εφαρμογής κατά τον χρόνο εκτέλεσης, παρακάμπτοντας οποιονδήποτε στατικό μηχανισμό προστασίας.

Η εργασία αποσκοπεί στο να αναδείξει ότι, σε περιβάλλοντα όπου ο κώδικας εκτελείται στη συσκευή του χρήστη, οι τεχνικές «ασφάλειας μέσω της απόκρυψης» (security through obscurity) παραμένουν ανεπαρκείς, υπογραμμίζοντας την αναγκαιότητα για υιοθέτηση ισχυρών πρακτικών server-side επαλήθευσης.

Στη συνέχεια, το αρχείο εγκατάστασης project02.apk εισήχθη στο περιβάλλον του εργαλείου προκειμένου να ξεκινήσει η αυτόματη διαδικασία αποσυμπίεσης και αποσφαλμάτωσης (decompilation) του κώδικα.



Μέσω του μενού πλοήγησης, η ανάλυση επικεντρώθηκε στον κατάλογο του πηγαίου κώδικα (Source code) και συγκεκριμένα στο πακέτο της εφαρμογής com.unipr.project02. Εκεί εντοπίστηκε η κύρια δραστηριότητα (activity) της εφαρμογής με την ονομασία **MainActivity**.

Εξετάζοντας τον παραγόμενο κώδικα Java της κλάσης MainActivity, εντοπίστηκε η μέθοδος login(), η οποία είναι υπεύθυνη για τον έλεγχο ταυτοποίησης του χρήστη κατά το πάτημα του κουμπιού σύνδεσης. Μέσω της ανάλυσης της μεθόδου, αποκαλύφθηκε πλήρως η λογική λειτουργίας του μηχανισμού:

- **Έλεγχος username:** Η εφαρμογή ανακτά το κείμενο που εισάγει ο χρήστης στο πεδίο uname και το συγκρίνει (μέσω της εντολής .equals()) με τη σταθερά (hardcoded string) «user457».
- **Έλεγχος password:** Παράλληλα, ανακτά τον κωδικό από το πεδίο pass και τον συγκρίνει με την τιμή που επιστρέφει η κλήση της native συνάρτησης stringFromJNI().
- Εφόσον οι συνθήκες ταυτίζονται (if block), η εφαρμογή επιτρέπει την είσοδο, προβάλλοντας το μήνυμα επιτυχίας «Login Successful!» μέσω μιας ειδοποίησης toast.

- Εάν η σύγκριση αποτύχει (else block), προβάλλεται το μήνυμα «Wrong Credentials!» και η είσοδος απορρίπτεται.

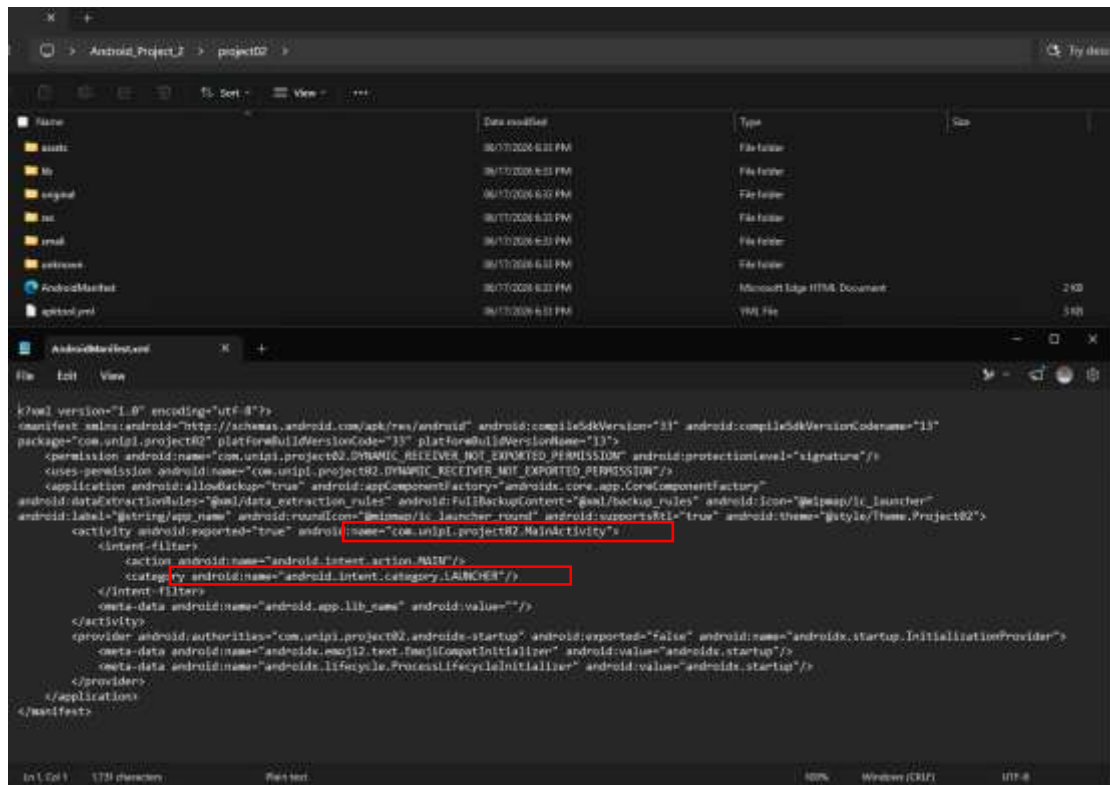
Λαμβάνοντας υπόψη ότι ο στόχος της μεθόδου 1 είναι η παράκαμψη του ελέγχου χωρίς τη χρήση του σωστού κωδικού (ο οποίος άλλωστε παράγεται δυναμικά από τη βιβλιοθήκη JNI), η προτεινόμενη λύση είναι το **Smali Patching**. Η τεχνική αυτή περιλαμβάνει την άμεση τροποποίηση του κώδικα μηχανής (Smali) ώστε η εντολή σύγκρισης να αντιστραφεί. Αλλάζοντας τη λογική συνθήκη της συνάρτησης ώστε να θεωρεί έγκυρα τα *λανθασμένα* στοιχεία, επιτυγχάνεται η πλήρης παράκαμψη του μηχανισμού.

Προκειμένου να πραγματοποιηθεί η αλλαγή (patching) στον κώδικα, ήταν απαραίτητη η εξαγωγή των συμπιεσμένων αρχείων της εφαρμογής. Για τον σκοπό αυτό, χρησιμοποιήθηκε το εργαλείο ανοιχτού κώδικα **APKTool**, το οποίο επιτρέπει την αποσυναρμολόγηση του εκτελέσιμου αρχείου .apk και την αναπαράσταση του κώδικα σε μορφή Smali.

Η διαδικασία εκτελέστηκε μέσω της διεπαφής γραμμής εντολών (Command Prompt), χρησιμοποιώντας την εντολή **apktool d project02.apk**.

```
C:\Users\Kakai\Desktop\Android_Project_2>apktool d project02.apk
I: Using Apktool 3.0.1 on project02.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: C:\Users\Kakai\AppData\Local\apktool\framework\1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying assets...
I: Copying lib...
I: Copying unknown files...
C:\Users\Kakai\Desktop\Android_Project_2>
```

Με την ολοκλήρωση της εκτέλεσης της εντολής, δημιουργήθηκε ένας νέος τοπικός κατάλογος (project02) που περιείχε τον πηγαίο κώδικα (σε μορφή Smali), τους πόρους (resources) και τα βασικά αρχεία ρυθμίσεων της εφαρμογής. Πριν την άμεση αναζήτηση της μεθόδου ελέγχου στους φακέλους της εφαρμογής, εξετάστηκε το αρχείο ρυθμίσεων AndroidManifest.xml (το οποίο ανοίχτηκε με απλό επεξεργαστή κειμένου), προκειμένου να χαρτογραφηθεί η δομή της εφαρμογής και να εντοπιστεί η κύρια δραστηριότητα (Main Activity).



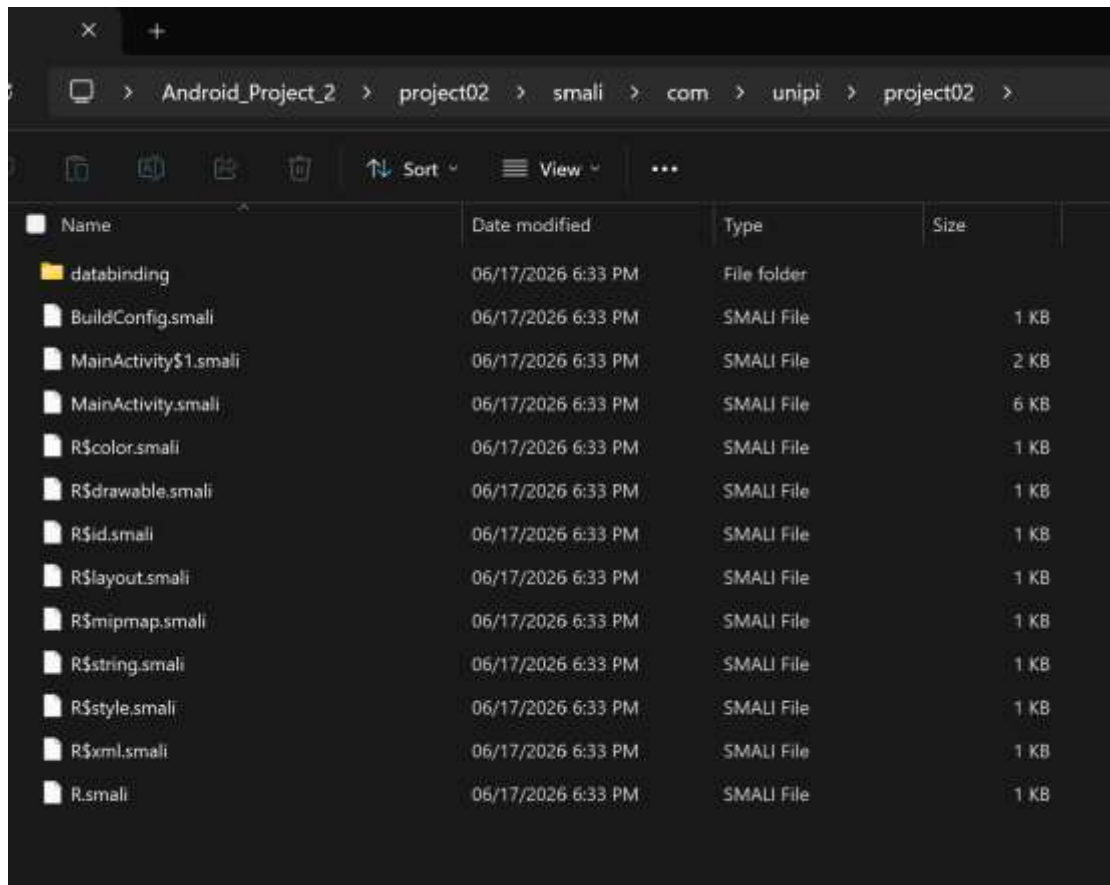
Από την επιθεώρηση του κώδικα XML, εξήχθησαν δύο κρίσιμες πληροφορίες:

1. Το φίλτρο πρόθεσης (intent-filter) που περιείχε την κατηγορία «<category android:name="android.intent.category.LAUNCHER"/>» επιβεβαίωσε ότι η συγκεκριμένη δραστηριότητα αποτελεί την αρχική οθόνη που φορτώνεται κατά την εκκίνηση της εφαρμογής (οθόνη Login).
2. Η δήλωση «android:name="com.unipi.project02.MainActivity"» αποκάλυψε την ακριβή διαδρομή και το όνομα του αρχείου που περιέχει τον κώδικα της οθόνης σύνδεσης.

Αξιοποιώντας αυτή την πληροφορία, η πλοήγηση κατευθύνθηκε στοχευμένα στη διαδρομή `project02/smali/com/unipi/project02/`, παρακάμπτοντας άσχετους καταλόγους συστήματος και έτοιμων βιβλιοθηκών (όπως `androidx` ή `kotlin`). Μέσα στον φάκελο `project02`, εντοπίστηκε το αρχείο `MainActivity.smali`. Το αρχείο αυτό ανοίχτηκε με επεξεργαστή κειμένου και μέσω αναζήτησης, εντοπίστηκε με ακρίβεια η συνάρτηση ελέγχου ταυτοποίησης που είχε ήδη παρατηρηθεί στο JADX, φέροντας την υπογραφή `.method public login()V`.

Μέσα στον κατάλογο του πηγαίου κώδικα `project02/smali/com/unipi/project02/` εντοπίζονται πολλαπλά αρχεία που σχετίζονται

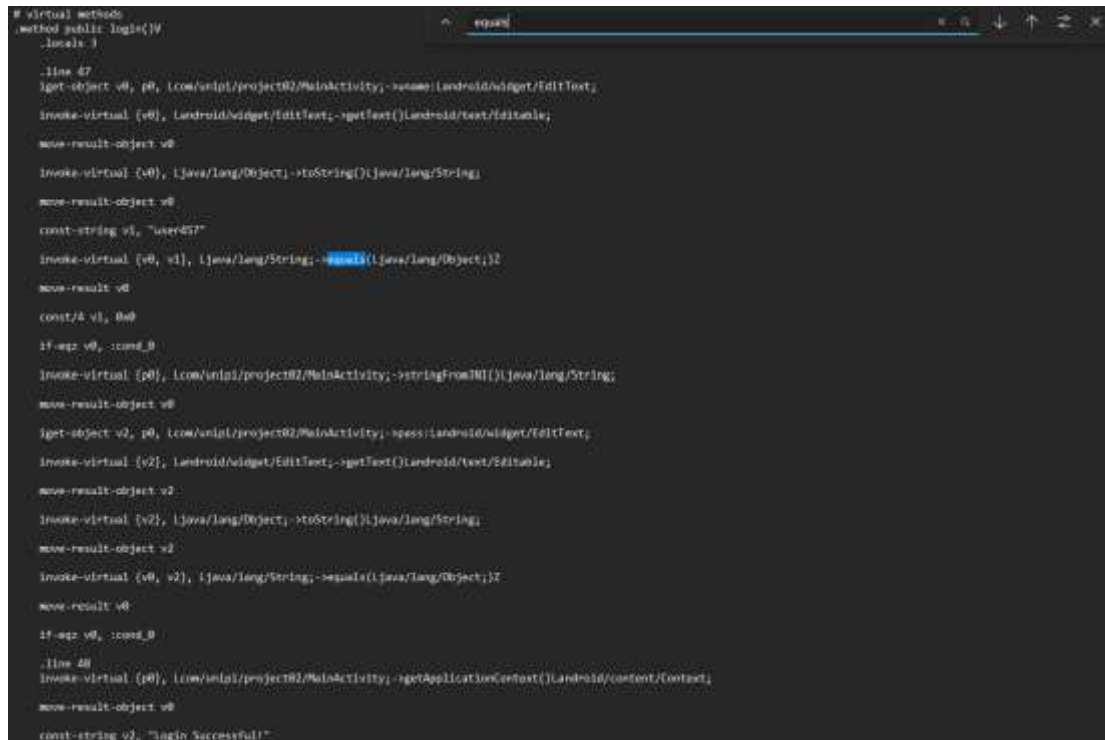
με το MainActivity (π.χ., MainActivity\$1.smali). Τα αρχεία που φέρουν το σύμβολο \$ αντιπροσωπεύουν ανώνυμες υπο-κλάσεις που δημιουργούνται κατά τη μεταγλώττιση, συνήθως για τη διαχείριση συμβάντων (όπως τα click listeners των κουμπιών). Η εξέταση αυτών των υπο-κλάσεων απέδειξε ότι απλώς ανακατευθύνουν την εκτέλεση στην κύρια μέθοδο login()V. Επομένως, η κύρια εστίαση της ανάλυσης παρέμεινε στο βασικό αρχείο MainActivity.smali.



Στο εσωτερικό του MainActivity.smali, η ροή εκτέλεσης της login()V καθορίζεται από δύο διαδοχικούς λογικούς ελέγχους:

1. **Έλεγχος username:** Αρχικά, η εφαρμογή καλεί τη μέθοδο equals (η οποία στον κώδικα Dalvik μεταφράζεται ως invoke-virtual) για να συγκρίνει το όνομα χρήστη που εισήχθη με την προκαθορισμένη (hardcoded) τιμή «user457». Το αποτέλεσμα αυτής της σύγκρισης αξιολογείται αμέσως μετά από την εντολή υπό όρο if-eqz (If Equal to Zero). Εάν τα στοιχεία δεν ταυτίζονται (η σύγκριση επιστρέφει 0/ψευδές), η εντολή αυτή ανακατευθύνει βίαια τη ροή εκτέλεσης στην ετικέτα «:cond_0», η οποία οδηγεί στην απόρριψη της εισόδου και την εμφάνιση μηνύματος σφάλματος.

2. **Έλεγχος password:** Εάν ο πρώτος έλεγχος ολοκληρωθεί με επιτυχία, το πρόγραμμα προχωρά στον έλεγχο του κωδικού. Εδώ, ακολουθείται η ίδια ακριβώς προγραμματιστική λογική (equals ακολουθούμενη από if-eqz), με τη διαφορά ότι το κείμενο του χρήστη συγκρίνεται με την τιμή που παράγει και επιστρέφει δυναμικά η εξωτερική (native) συνάρτηση `stringFromJNI()`. Ομοίως, εάν οι κωδικοί δεν ταιριάζουν, η εκτέλεση εκτρέπεται ξανά στην ετικέτα αποτυχίας «:cond_0».



```
# virtual methods
.method public login()V
    .locals 3

    .line 47
    iget-object v0, p0, Lcom/example/project02/MainActivity; ->username:Landroid/widget/EditText;
    invoke-virtual {v0, Landroid/widget/EditText; ->getText()Landroid/text/Editable;
    move-result-object v0

    invoke-virtual {v0, Ljava/lang/Object; ->toString()Ljava/lang/String;
    move-result-object v0

    const-string v1, "user457"
    invoke-virtual {v0, v1, Ljava/lang/String; ->equals(Ljava/lang/Object;)Z
    move-result v0

    const/4 v1, 0x0
    if-eqz v0, :cond_0

    invoke-virtual {p0, Lcom/example/project02/MainActivity; ->stringFromJNI()Ljava/lang/String;
    move-result-object v0

    iget-object v2, p0, Lcom/example/project02/MainActivity; ->password:Landroid/widget/EditText;
    invoke-virtual {v2, Landroid/widget/EditText; ->getText()Landroid/text/Editable;
    move-result-object v2

    invoke-virtual {v2, Ljava/lang/Object; ->toString()Ljava/lang/String;
    move-result-object v2

    invoke-virtual {v0, v2, Ljava/lang/String; ->equals(Ljava/lang/Object;)Z
    move-result v0

    if-eqz v0, :cond_0

    .line 48
    invoke-virtual {p0, Lcom/example/project02/MainActivity; ->getApplicationContext()Landroid/content/Context;
    move-result-object v0

    const-string v2, "Login Successful!"
```

Για την πλήρη παράκαμψη αυτού του μηχανισμού, εφαρμόστηκε η τεχνική του **Smali Patching**. Αυτή συνίσταται στην αντικατάσταση των εντολών «if-eqz» με το λογικό τους αντίθετο, τις εντολές «if-nez» (If Not Equal to Zero). Με αυτή την αλλαγή λειτουργικού κώδικα, η σημασία κάθε ελέγχου αντιστρέφεται. Πλέον κάθε εντολή «if-nez» εκτρέπει τη ροή προς την ετικέτα αποτυχίας («:cond_0») όταν το αντίστοιχο πεδίο ταυτίζεται με την πραγματική τιμή, και όχι όταν διαφέρει από αυτήν. Συνεπώς, η διαδρομή επιτυχίας («Login Successful!») προσεγγίζεται μόνο όταν και τα δύο πεδία (Username και Password) διαφέρουν ταυτόχρονα από τα αυθεντικά διαπιστευτήρια. Εισάγοντας, επομένως, εσκεμμένα λανθασμένες τιμές και στα δύο πεδία, επιτυγχάνεται η είσοδος χωρίς να απαιτείται η γνώση του πραγματικού

κωδικού, ο οποίος, μετά την τροποποίηση, οδηγεί πλέον ο ίδιος σε απόρριψη.

```
# virtual methods
.method public login()V
    .locals 3

    .line 47
    iget-object v0, p0, Lcom/unipi/project02/MainActivity;.>uname:Landroid/widget/EditText;

    invoke-virtual {v0}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;

    move-result-object v0

    invoke-virtual {v0}, Ljava/lang/Object;.>toString()Ljava/lang/String;

    move-result-object v0

    const-string v1, "user457"

    invoke-virtual {v0, v1}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z

    move-result v0

    const/4 v1, 0x0

    if-nez v0, :cond_0

    invoke-virtual {p0, Lcom/unipi/project02/MainActivity;.>getStringFromME()Ljava/lang/String;

    move-result-object v0

    iget-object v2, p0, Lcom/unipi/project02/MainActivity;.>pass:Landroid/widget/EditText;

    invoke-virtual {v2}, Landroid/widget/EditText;.>getText()Landroid/text/Editable;

    move-result-object v2

    invoke-virtual {v2}, Ljava/lang/Object;.>toString()Ljava/lang/String;

    move-result-object v2

    invoke-virtual {v0, v2}, Ljava/lang/String;.>equals(Ljava/lang/Object;)Z

    move-result v0

    if-nez v0, :cond_0

    .line 48
    invoke-virtual {p0}, Lcom/unipi/project02/MainActivity;.>getApplicationContext()Landroid/content/Context;

    move-result-object v0

    const-string v2, "Login Successful"
```

Ολοκληρώνοντας την τροποποίηση του κώδικα Smali, το επόμενο στάδιο απαιτεί την ανακατασκευή του αποσυμπιεσμένου καταλόγου σε ένα νέο, εκτελέσιμο αρχείο μορφής APK. Η διαδικασία αυτή πραγματοποιήθηκε μέσω του τερματικού, εκτελώντας την εντολή **apktool b project02 -o project02_patched.apk**. Η παράμετρος «b» υποδηλώνει τη διαδικασία δόμησης (build), ενώ η παράμετρος «-o» καθορίζει το όνομα του εξαγόμενου αρχείου (project02_patched.apk), διασφαλίζοντας ότι το αρχικό APK παραμένει ανέπαφο. Η επιτυχής εκτέλεση της εντολής επιβεβαιώθηκε από την απουσία σφαλμάτων και τη δημιουργία του νέου αρχείου στον κατάλογο εργασίας.

```
C:\Users\Kalai\Desktop\Android_Project_2>apktool b project02 -o project02_patched.apk
I: Using Apktool 3.0.1 on project02.apk with 8 threads
I: Smaling smali folder into classes.dex...
I: AndroidManifest.xml and resources have not changed.
I: Building apk file...
I: Importing assets...
I: Importing lib...
I: Importing unknown files...
I: Built apk into: project02_patched.apk

C:\Users\Kalai\Desktop\Android_Project_2>
```

project02	06/17/2026 8:30 PM	File folder
Writeup_Template	06/12/2026 12:30 AM	File folder
apktool	06/17/2026 8:30 PM	Windows Batch File
apktool	06/17/2026 8:30 PM	Executable Jar File
project02.apk	06/12/2026 12:30 AM	APK File
project02_patched.apk	06/17/2026 8:33 PM	APK File
README	06/12/2026 12:30 AM	Text Document

Παρά την επιτυχή ανακατασκευή, η απευθείας εγκατάσταση του αρχείου project02_patched.apk στο περιβάλλον του Android Studio καθίσταται αδύνατη. Το λειτουργικό σύστημα Android ενσωματώνει αυστηρούς μηχανισμούς ασφαλείας, απαγορεύοντας ρητά την εγκατάσταση εφαρμογών (packages) που δεν φέρουν έγκυρη ψηφιακή υπογραφή. Δεδομένου ότι το Apktool ανακατασκευάζει το αρχείο χωρίς να το υπογράφει, απαιτήθηκε η χρήση ενός εξωτερικού εργαλείου υπογραφής.

Για τον σκοπό αυτό, αξιοποιήθηκε το αυτόνομο εργαλείο **uber-apk-signer**. Μέσω της εκτέλεσης της εντολής **java -jar uber-apk-signer-1.3.0.jar -a project02_patched.apk**, το εργαλείο ανέλαβε την αυτόματη ευθυγράμμιση (zipalign) και την ψηφιακή υπογραφή του αρχείου. Το αποτέλεσμα της διαδικασίας ήταν η παραγωγή του τελικού, λειτουργικού και ψηφιακά υπογεγραμμένου πακέτου με την ονομασία project02_patched-aligned-debugSigned.apk.

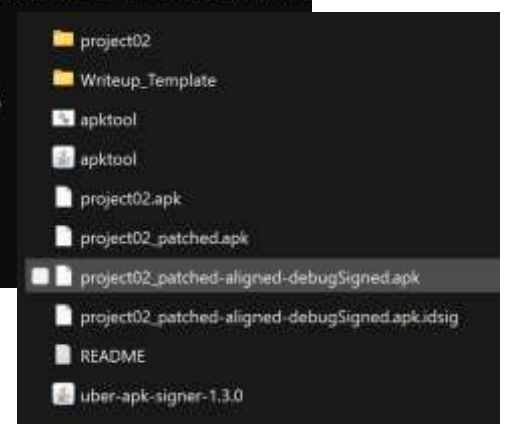
```
C:\Users\Walai\Desktop\Android_Project_2>java -jar uber-apk-signer-1.3.0.jar -a project02_patched.apk
source: C:\Users\Walai\Desktop\Android_Project_2
binary-lib/windows-33_0_2/libwinpthread-1.dll
C:\Users\Walai\AppData\Local\Temp\uspsigner-7386785299392876380
zipalign location: BUILD_TOOLS
C:\Users\Walai\AppData\Local\Temp\uspsigner-7386785299392876380\win-zipalign_33_0_2.exe16418115412612769117.tmp
keystore: [0] e8a8f3d2 C:\Users\Walai\.android\debug.keystore (DEBUG_ANDROID_FOLDER)

01. project02_patched.apk

SIGN
file: C:\Users\Walai\Desktop\Android_Project_2\project02_patched.apk (4.78 MiB)
checksum: 1110b625cbfa0dd124a88d931c7e5e2f38dab306d4038e9925713a0673edeb (sha256)
- zipalign success
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::loadLibrary has been called by org.conscrypt.NativeLibraryUtil in an unnamed module (file:/C:/Users/Walai/Desktop/Android_Project_2/uber-apk-signer-1.3.0.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled
- sign success

VERIFY
file: C:\Users\Walai\Desktop\Android_Project_2\project02_patched-aligned-debugSigned.apk (4.88 MiB)
checksum: d64d88104d28e61cc2f3f1f474a5290a893fe76f226c3a7c9c6857d77736148b (sha256)
- zipalign verified
- signature verified [v1, v2, v3]
45 warnings
Subject: C=US, O=Android, CN=Android Debug
SHA256: #f963756f20f12b42062539f331206c3d5aach567c6586f635ba5d2ccdbff7ea / SHA256withRSA
Expires: Sat Apr 22 11:44:24 EEST 2026

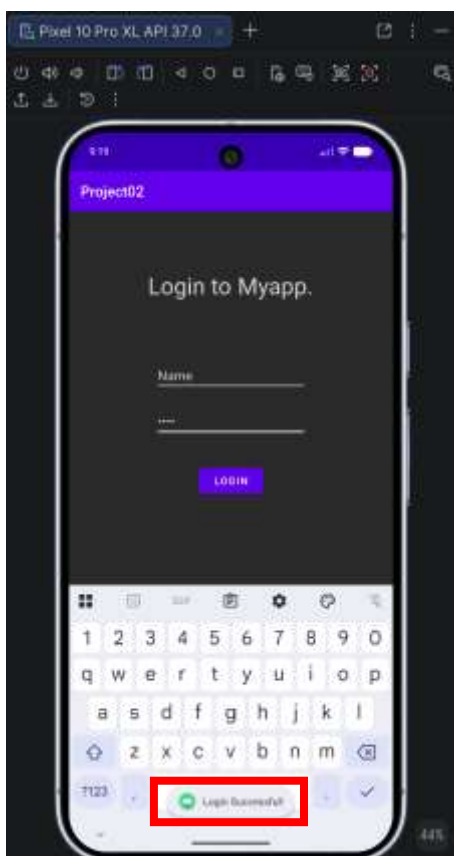
[Wed Jun 29 20:50:13 EEST 2026][v1.3.0]
Successfully processed 1 APKs and 0 errors in 1.11 seconds.
C:\Users\Walai\Desktop\Android_Project_2>
```



Το τελικό υπογεγραμμένο αρχείο project02_patched-aligned-debugSigned.apk εγκαταστάθηκε στην εικονική συσκευή (Android Emulator) μέσω διαδικασίας drag & drop. Κατά την εκτέλεση της εφαρμογής, πραγματοποιήθηκε δοκιμή ταυτοποίησης

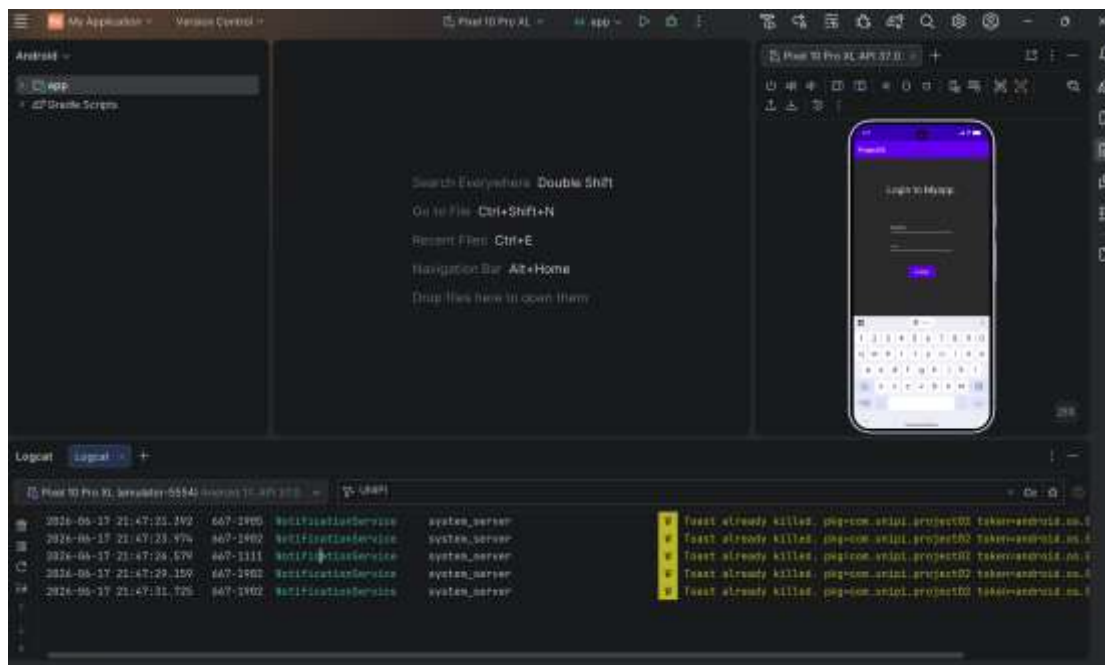
εισάγοντας εσκεμμένα λανθασμένα και τυχαία στοιχεία (π.χ., κενό στο πεδίο Username και «1234» στο πεδίο Password). Με την υποβολή των στοιχείων, η εφαρμογή αξιολόγησε τα διαπιστευτήρια.

Λόγω της προηγηθείσας αναστροφής των λογικών ελέγχων (if-eqz σε if-nez) εντός του κώδικα Smali, η σημασία των συγκρίσεων αντιστράφηκε, με αποτέλεσμα η έγκριση της εισόδου να απαιτεί πλέον στοιχεία που διαφέρουν από τα πραγματικά και στα δύο πεδία. Για τον λόγο αυτό, η δοκιμή πραγματοποιήθηκε με κενό πεδίο Username και τον τυχαίο κωδικό «1234» -τιμές που διαφέρουν αμφότερες από τα αυθεντικά διαπιστευτήρια- οδηγώντας στην εμφάνιση του μηνύματος «Login Successful!» και επιβεβαιώνοντας την παράκαμψη χωρίς τη χρήση του πραγματικού κωδικού.



Κατά την επιτυχή εκτέλεση της παράκαμψης, παρατηρήθηκε ότι η εφαρμογή δεν εμφανίζει το ζητούμενο flag (της μορφής UNIP1{...}) στην οθόνη της συσκευής, παρά μόνο το μήνυμα επιτυχίας. Προκειμένου να διερευνηθεί το ενδεχόμενο το flag να εκτυπώνεται στο παρασκήνιο κατά τη διαδικασία της ταυτοποίησης, πραγματοποιήθηκε ανάλυση των αρχείων καταγραφής συστήματος (logs) μέσω της καρτέλας Logcat του Android Studio.

Εφαρμόζοντας φίλτρο αναζήτησης με τη λέξη-κλειδί «UNIPi», τα μόνα ευρήματα αφορούσαν ειδοποιήσεις συστήματος (π.χ., «Toast already killed»), οι οποίες καταγράφηκαν αποκλειστικά λόγω της ονομασίας του πακέτου της εφαρμογής (com.unipi.project02).



Όπως επιβεβαιώθηκε και από τον πηγαίο κώδικα Java της εφαρμογής (στο JADX), ο μηχανισμός απλώς εκτυπώνει το μήνυμα ειδοποίησης (Toast) χωρίς να διαχειρίζεται το κείμενο του flag. Συνεπώς, η μέθοδος 1 ολοκληρώθηκε με απόλυτη επιτυχία, καθώς επιτεύχθηκε ο πρωταρχικός στόχος του πλήρους Login Bypass χωρίς τη γνώση του κωδικού πρόσβασης.

Μέθοδος 2: JADX και Ghidra (Εξαγωγή διαπιστευτηρίων μέσω αντίστροφης μηχανικής)

Για την εκκίνηση της δεύτερης μεθόδου, η οποία στοχεύει στην πραγματική ανάκτηση του κρυμμένου κωδικού πρόσβασης, απαιτήθηκε η επαναφορά του περιβάλλοντος ελέγχου. Η τροποποιημένη εφαρμογή (patched) απεγκαταστάθηκε από την εικονική συσκευή, και στη θέση της εγκαταστάθηκε εκ νέου το αυθεντικό, μη τροποποιημένο αρχείο project02.apk.

Κατά την αρχική στατική ανάλυση με το εργαλείο JADX, είχε εντοπιστεί εντός της κλάσης MainActivity η δήλωση «System.loadLibrary("project02");», καθώς και η κλήση της εξωτερικής συνάρτησης «stringFromJNI()». Αυτό το εύρημα υποδεικνύει

ότι ο προγραμματιστής, για λόγους αυξημένης ασφάλειας έναντι της αποσυναρμολόγησης επιπέδου Dalvik, υλοποίησε τον μηχανισμό επαλήθευσης του κωδικού σε μια εξωτερική, native βιβλιοθήκη C/C++ (Shared Object). Ο πραγματικός κωδικός πρόσβασης (Flag) βρίσκεται ενσωματωμένος (hardcoded) στο εσωτερικό της. Προκειμένου να ανακτηθεί η εν λόγω βιβλιοθήκη, το αρχείο project02.apk αντιμετωπίστηκε ως συμπιεσμένος φάκελος μορφής .zip και ανοίχτηκε με τη χρήση λογισμικού αποσυμπίεσης (7-Zip). Πλοηγούμενοι στη διαδρομή lib/x86_64/ (κατάλογος που περιέχει τις μεταγλωττισμένες βιβλιοθήκες για την αρχιτεκτονική του εικονικού επεξεργαστή), εντοπίστηκε το αρχείο libproject02.so.

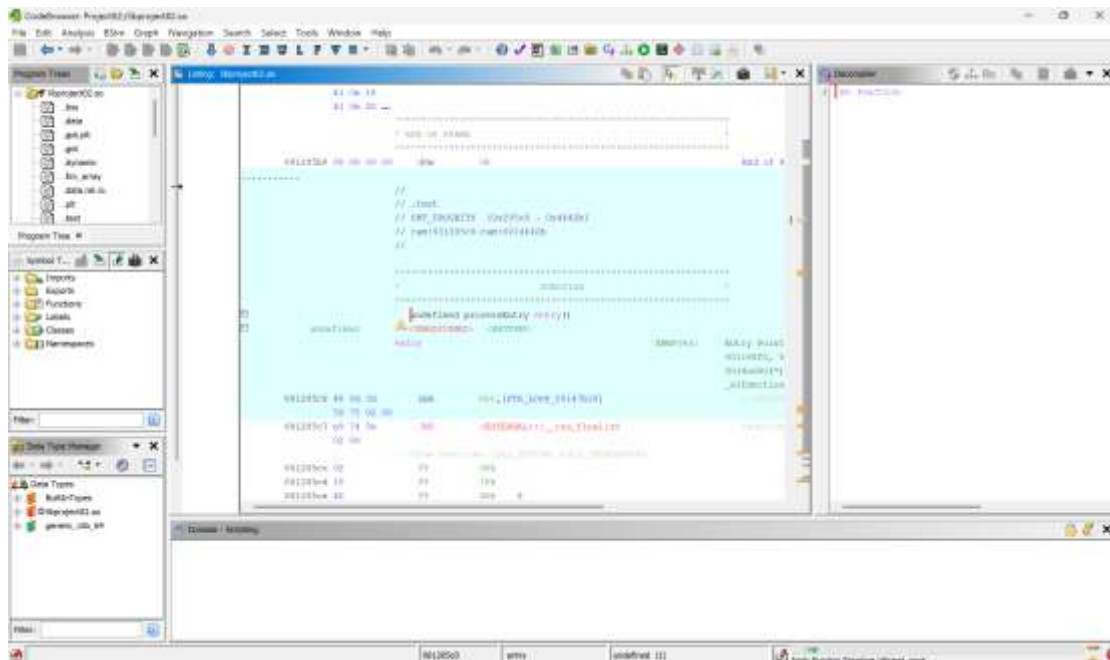


Το αρχείο αυτό εξήχθη στον τοπικό κατάλογο εργασίας, αποτελώντας πλέον τον κύριο στόχο της ανάλυσης. Για την ανάλυση του μεταγλωττισμένου κώδικα μηχανής της βιβλιοθήκης, αξιοποιήθηκε το εξειδικευμένο εργαλείο αντίστροφης μηχανικής **Ghidra**. Το αρχείο libproject02.so εισήχθη σε ένα νέο Project του εργαλείου.

Κατά την εισαγωγή, το σύστημα αναγνώρισε επιτυχώς τη δομή του αρχείου ως 64-bit ELF (Executable and Linkable Format), βελτιστοποιημένο για την αρχιτεκτονική x86, ολοκληρώνοντας τη διαδικασία χωρίς σφάλματα, όπως επιβεβαιώθηκε από το τελικό πόρισμα (Import Results Summary).

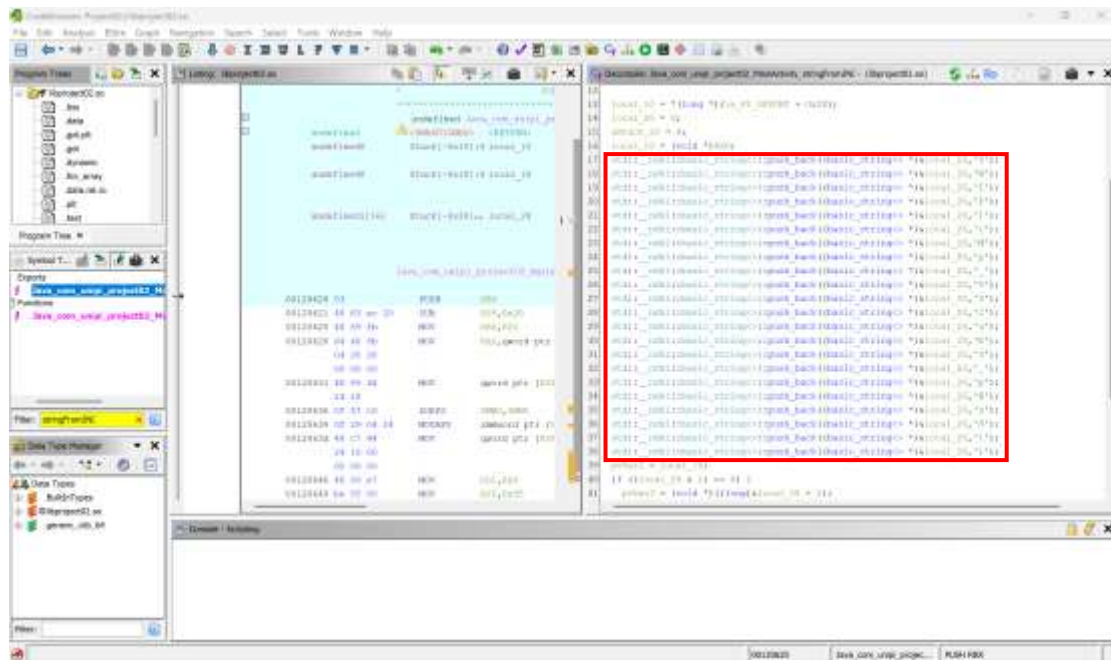


Με το άνοιγμα του αρχείου libproject02.so στο κύριο περιβάλλον εργασίας του Ghidra, εκτελέστηκε η διαδικασία της αυτόματης ανάλυσης διατηρώντας τις προεπιλεγμένες ρυθμίσεις του εργαλείου, προκειμένου να χαρτογραφηθούν οι συναρτήσεις και οι δομές δεδομένων της βιβλιοθήκης.



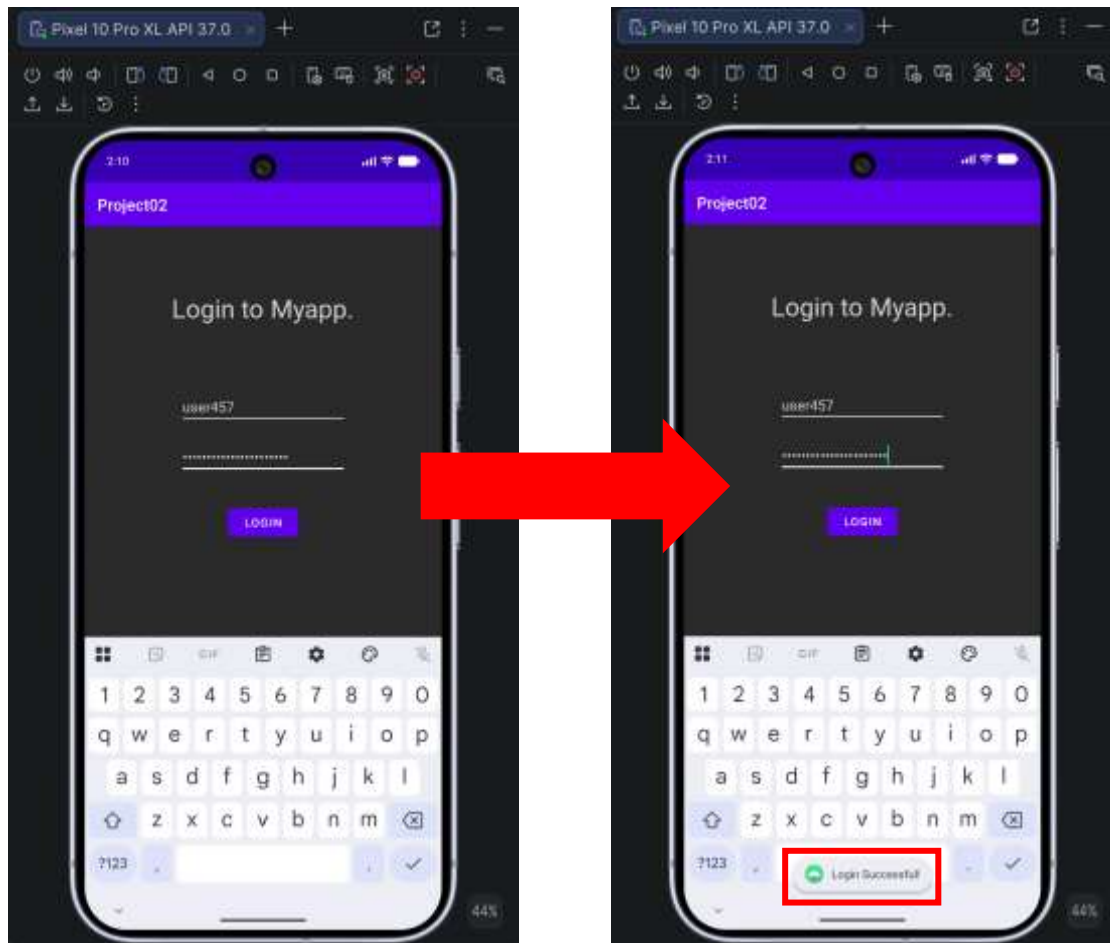
Με βάση τα ευρήματα της στατικής ανάλυσης από το JADX (μέθοδος 1), ήταν γνωστό ότι η εφαρμογή καλεί τη native μέθοδο `stringFromJNI()`. Κάνοντας χρήση του παραθύρου Symbol Tree και εφαρμόζοντας το αντίστοιχο φίλτρο αναζήτησης στις συναρτήσεις (Functions), εντοπίστηκε η εν λόγω συνάρτηση, η οποία κατά τη μεταγλώττιση είχε λάβει την πλήρη υπογραφή «`Java_com_unipi_project02_MainActivity_stringFromJNI`». Επιλέγοντας τη συνάρτηση-στόχο, το εργαλείο προχώρησε σε αποσφαλμάτωση, μεταφράζοντας τον κώδικα μηχανής σε αναγνώσιμη μορφή C/C++ στο αντίστοιχο παράθυρο.

Από την εξέταση του πηγαιού κώδικα, διαπιστώθηκε ότι ο δημιουργός της εφαρμογής εφάρμοσε τεχνικές απόκρυψης για την προστασία του κωδικού. Ειδικότερα, χρησιμοποιήθηκε η τεχνική των stack strings, όπου το επιθυμητό αλφαριθμητικό δεν αποθηκεύεται ως μια ενιαία, συνεχόμενη μεταβλητή (η οποία θα ήταν εύκολα ανιχνεύσιμη με εργαλεία όπως το strings), αλλά κατασκευάζεται δυναμικά στη μνήμη κατά τον χρόνο εκτέλεσης. Αυτό επιτυγχάνεται μέσω διαδοχικών κλήσεων της μεθόδου `push_back` της κλάσης `std::__ndk1::basic_string`, εισάγοντας τον κωδικό χαρακτήρα προς χαρακτήρα.



Ενοποιώντας τους μεμονωμένους χαρακτήρες με τη σειρά εκτέλεσής τους στον κώδικα, το εργαλείο του Ghidra αποκρυπτογράφησε πλήρως την αλληλουχία, αποκαλύπτοντας το πραγματικό διαπιστευτήριο και συνάμα το ζητούμενο flag της άσκησης: **UNIPi{My_53cuR3_p@sS!}**

Για την οριστική επαλήθευση της μεθόδου 2, το αρχικό, μη-τροποποιημένο αρχείο project02.apk εκτελέστηκε στον προσομοιωτή Android. Εισάγοντας το hardcoded όνομα χρήστη (**user457**) και τον κωδικό που μόλις ανακτήθηκε από το Ghidra (**UNIPi{My_53cuR3_p@sS!}**), η εφαρμογή επέτρεψε την είσοδο, εμφανίζοντας το μήνυμα επιτυχίας.

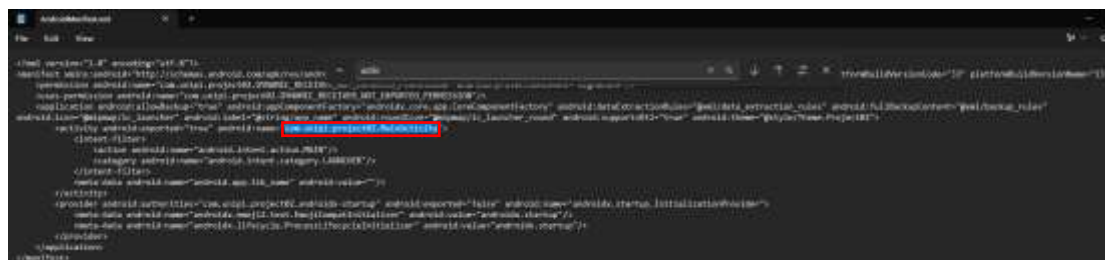


Σύμφωνα με τις απαιτήσεις της άσκησης στο αρχείο README.txt, ζητείται η καταγραφή τριών διαφορετικών προσεγγίσεων για την παράκαμψη του μηχανισμού σύνδεσης. Έχοντας ήδη ολοκληρώσει με επιτυχία το Smali Patching και το Native Reverse Engineering, διερευνήθηκε η πιθανότητα μιας εναλλακτικής στατικής μεθόδου, όπως η παραποίηση των προθέσεων (Activity/Intent Manipulation).

Υπόθεση Manifest Bypass

Μια συνήθης τεχνική στατική παράκαμψης περιλαμβάνει την τροποποίηση του αρχείου AndroidManifest.xml (μέσω του Apktool). Αν η εφαρμογή διέθετε μια ξεχωριστή, προστατευμένη οθόνη (π.χ. SecondActivity), θα ήταν εφικτή η μεταφορά του φίλτρου πρόθεσης «<category android:name="android.intent.category.LAUNCHER"/>» σε αυτήν, εξαναγκάζοντας το λειτουργικό σύστημα να φορτώσει απευθείας την εσωτερική οθόνη, προσπερνώντας εντελώς την οθόνη ταυτοποίησης (Login). Ωστόσο, κατά την επιθεώρηση του αποσυμπίεσμένου AndroidManifest.xml, διαπιστώθηκε ότι η εφαρμογή υλοποιεί την

αρχιτεκτονική της αποκλειστικά σε μία ενιαία δραστηριότητα (tag <activity>): την com.unipi.project02.MainActivity.



Η απουσία εναλλακτικών δραστηριοτήτων καταρρίπτει τεχνικά τη δυνατότητα του Manifest Bypass. Η εφαρμογή δεν δρομολογεί τον χρήστη σε νέα οθόνη, αλλά εκτελεί τον έλεγχο και αποδίδει το αποτέλεσμα (μέσω Toast μηνύματος) στο ίδιο περιβάλλον. Βάσει του αποκλεισμού αυτής της στατικής μεθόδου, συνάγεται το συμπέρασμα ότι η ενδεδειγμένη τρίτη προσέγγιση (η οποία δεν αναφέρεται στα tips της άσκησης) μεταβαίνει στο πεδίο της δυναμικής ανάλυσης. Η χρήση εργαλείων όπως το **Frida** θα επέτρεπε την παρέμβαση στη μνήμη της συσκευής κατά τον χρόνο εκτέλεσης, αντικαθιστώντας δυναμικά την τιμή επιστροφής της μεθόδου login() ή υποκλέπτοντας τον κωδικό από τη μνήμη RAM πριν τη διαδικασία της σύγκρισης.

Μέθοδος 3: Δυναμική ανάλυση με το Frida.

Έχοντας εξετάσει τις στατικές προσεγγίσεις παρέμβασης (Smali Patching και Ghidra Decompilation), η τρίτη μέθοδος εστιάζει στον πυλώνα της δυναμικής ανάλυσης. Σε αυτή την προσέγγιση, δεν επιδιώκεται η τροποποίηση του φυσικού εκτελέσιμου αρχείου (APK), αλλά η άμεση επέμβαση στη μνήμη (RAM) της συσκευής και στην ακολουθία εκτέλεσης των οδηγιών κατά τον χρόνο λειτουργίας (runtime) της εφαρμογής. Για την υλοποίηση αυτής της επίθεσης επελέγη το κορυφαίο πλαίσιο ανοιχτού κώδικα **Frida**.

Βασική προϋπόθεση για την εκκίνηση της δυναμικής ανάλυσης αποτελεί η εγκατάσταση της αυθεντικής (μη-τροποποιημένης) έκδοσης της εφαρμογής (project02.apk) σε μια εικονική συσκευή (Android Emulator). Το εργαλείο Frida βασίζεται σε αρχιτεκτονική πελάτη-διακομιστή (client-server), γεγονός που καθιστά αναγκαία τη φιλοξενία και συνεχή εκτέλεση ενός αυτόνομου αρχείου (frida-server) στο παρασκήνιο του λειτουργικού συστήματος της συσκευής-στόχου. Καθώς το εργαλείο αυτό αλληλεπιδρά άμεσα με τη μνήμη και τις διεργασίες τρίτων εφαρμογών, είναι

επιβεβλημένη η απόκτηση δικαιωμάτων διαχειριστή/υπερχρήστη (Root) στο σύστημα Android.

Κατά την αρχική απόπειρα κλιμάκωσης προνομίων μέσω του εργαλείου γραμμής εντολών Android Debug Bridge (ADB), με τη χρήση της εντολής `adb root`, το σύστημα απέρριψε το αίτημα επιστρέφοντας το σφάλμα «adb cannot run as root in production builds». Το σφάλμα αυτό αναδεικνύει έναν βασικό μηχανισμό ασφαλείας του οικοσυστήματος Android. Η εικονική συσκευή (Pixel 10 Pro XL) που χρησιμοποιήθηκε αρχικά, είχε δημιουργηθεί με εικόνα συστήματος (System Image) η οποία ενσωμάτωνε το Google Play Store. Η Google χαρακτηρίζει αυτές τις εικόνες ως Production Builds. Με στόχο την προσομοίωση μιας πραγματικής, εμπορικής συσκευής, το λειτουργικό σύστημα σε αυτές τις εκδόσεις είναι αυστηρά κλειδωμένο, αποτρέποντας την εκτέλεση του ADB (`adb`) με δικαιώματα root. Προκειμένου να παρακαμφθεί αυτός ο αρχιτεκτονικός περιορισμός, αποφασίστηκε η δημιουργία μιας νέας εικονικής συσκευής μέσω του Device Manager του Android Studio.

Το «κλειδί» της επιτυχίας σε αυτό το βήμα ήταν η στρατηγική επιλογή της κατάλληλης έκδοσης λογισμικού. Αντί για εικόνες με υποστήριξη Google Play, επιλέχθηκε μια εικόνα συστήματος βασισμένη αποκλειστικά σε **Google APIs** (αρχιτεκτονικής επεξεργαστή `x86_64`). Οι συγκεκριμένες εκδόσεις (Target Images) είναι παραμετροποιημένες, καθώς δεν φέρουν τους περιορισμούς παραγωγής και επιτρέπουν την ελεύθερη απόκτηση προνομίων Root. Αφού εγκαταστάθηκε η εφαρμογή `project02.apk` στη νέα, ανοιχτή εικονική συσκευή, εκτελέστηκε εκ νέου η εντολή κλιμάκωσης προνομίων παρέχοντας την πλήρη διαδρομή του εκτελέσιμου ADB: `C:\Users\Kalai\AppData\Local\Android\Sdk\platform-tools\adb root`. Η συγκεκριμένη προσπάθεια στέφθηκε με απόλυτη επιτυχία. Το σύστημα ανταποκρίθηκε άμεσα με το μήνυμα «restarting adb as root», υποδεικνύοντας ότι η υπηρεσία παρασκηνίου του Android Debug Bridge τερματίστηκε και επανεκκινήθηκε επιτυχώς ως διαδικασία υπερχρήστη, παρέχοντας πλέον απόλυτο έλεγχο στο αρχείο συστήματος (file system) και τη μνήμη του κινητού.



Προκειμένου το πλαίσιο Frida να αποκτήσει πρόσβαση στις διεργασίες του λειτουργικού συστήματος, απαιτείται η ύπαρξη του αντίστοιχου διακομιστή (frida-server) εντός της συσκευής. Βάσει της αρχιτεκτονικής του προσομοιωτή που δημιουργήθηκε (`x86_64`), πραγματοποιήθηκε λήψη της αντίστοιχης έκδοσης του

διακομιστή από το επίσημο αποθετήριο. Το αρχείο αποσυμπίεστηκε, μετονομάστηκε σε «frida-server» για ευκολία στη διαχείριση και τοποθετήθηκε στον κεντρικό κατάλογο εργασίας (Android_Project_2). Η διαδικασία εγκατάστασης και εκτέλεσης του διακομιστή στην εικονική συσκευή υλοποιήθηκε σε τρία διαδοχικά στάδια μέσω του τερματικού, αξιοποιώντας τις δυνατότητες του ADB:

1. **Μεταφορά του εκτελέσιμου αρχείου:** Η εντολή `adb push` χρησιμοποιήθηκε για την ασφαλή μεταφορά του αρχείου από το τοπικό σύστημα (host) στον προσωρινό κατάλογο της εικονικής συσκευής (`/data/local/tmp/`), ο οποίος χρησιμοποιείται παραδοσιακά για την εκτέλεση εξωτερικών εργαλείων.

```
C:\Users\Kala\\Desktop\Android_Project_2>C:\Users\Kala\AppData\Local\Android\Sdk\platform-tools\adb push frida-server /data/local/tmp/
frida-server: 1 file pushed, 0 skipped. 155.5 MB/s (111475776 bytes in 0.684s)
C:\Users\Kala\Desktop\Android_Project_2>
```

2. **Απόδοση δικαιωμάτων εκτέλεσης:** Δεδομένου ότι το λειτουργικό σύστημα Android βασίζεται στον πυρήνα του Linux, το μεταφερόμενο αρχείο δεν διέθετε αρχικά δικαιώματα εκτέλεσης. Μέσω της εντολής `adb shell "chmod 755 /data/local/tmp/frida-server"`, τροποποιήθηκαν τα δικαιώματα του αρχείου, επιτρέποντας στον χρήστη (owner) να το εκτελέσει.

```
C:\Users\Kala\Desktop\Android_Project_2>C:\Users\Kala\AppData\Local\Android\Sdk\platform-tools\adb shell "chmod 755 /data/local/tmp/frida-server"
C:\Users\Kala\Desktop\Android_Project_2>
```

3. **Εκκίνηση της υπηρεσίας στο παρασκήνιο:** Για να παραμείνει ο διακομιστής ενεργός χωρίς να δεσμεύει τη συνεδρία του τερματικού, εκτελέστηκε η εντολή `adb shell "/data/local/tmp/frida-server &"`. Η προσθήκη του συμβόλου «&» στο τέλος της εντολής εξασφαλίζει ότι η διεργασία θα τρέχει αθόρυβα στο παρασκήνιο. Η απουσία μηνυμάτων σφάλματος και η άμεση επιστροφή σε κενό prompt επιβεβαίωσαν την επιτυχή έναρξη λειτουργίας του διακομιστή.

```
C:\Users\Kala\Desktop\Android_Project_2>C:\Users\Kala\AppData\Local\Android\Sdk\platform-tools\adb shell "/data/local/tmp/frida-server &"
```

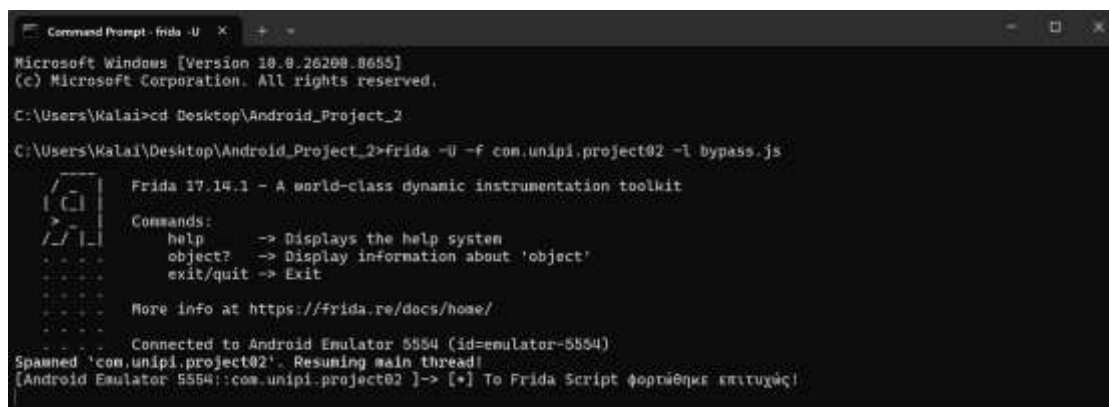
Με τον διακομιστή σε πλήρη ετοιμότητα, το επόμενο στάδιο αφορούσε τη συγγραφή του κώδικα ο οποίος θα πραγματοποιούσε την «ένεση» στη μνήμη της εφαρμογής. Για τον σκοπό αυτό, δημιουργήθηκε ένα εξειδικευμένο σενάριο (script) σε γλώσσα JavaScript με την ονομασία `bypass.js`. Προκειμένου να αναδειχθεί η πλήρης κατανόηση του μηχανισμού hooking, ο κώδικας σχεδιάστηκε με γνώμονα τη βέλτιστη

διαχείριση σφαλμάτων και την αναλυτική καταγραφή των ενεργειών (logging). Ο κώδικας που αναπτύχθηκε παρατίθεται παρακάτω:

```
if (Java.available) {  
    Java.perform(function () {  
        console.log("[+] Dynamic instrumentation environment initialized  
successfully.");  
  
        // Ανάκτηση της κλάσης της εφαρμογής από τη μνήμη  
        var targetActivity = Java.use("com.unipi.project02.MainActivity");  
  
        // Παράκαμψη της μεθόδου login() μέσω επανακαθορισμού της υλοποίησής της  
        targetActivity.login.implementation = function () {  
            console.log("[+] Intercepted call to: MainActivity.login()");  
            console.log("[+] Diverting execution flow to bypass hardcoded verification.");  
  
            // Δυναμική φόρτωση των απαραίτητων κλάσεων του Android SDK  
            var androidToast = Java.use("android.widget.Toast");  
            var javaString = Java.use("java.lang.String");  
  
            // Λήψη του context της τρέχουσας εφαρμογής (Application Context)  
            var currentContext = this.getApplicationContext();  
  
            // Κατασκευή και προβολή του προσαρμοσμένου μηνύματος επιτυχίας  
            var bypassNotification = javaString.$new("Login Successful! (Frida Bypass)");  
            androidToast.makeText(currentContext, bypassNotification, 0).show();  
  
            console.log("[+] Custom notification injected. Authentication controls  
bypassed.");  
        };  
    });  
}
```


Το σενάριο ελέγχει αρχικά τη διαθεσιμότητα του περιβάλλοντος Java (Java.available) και δημιουργεί ένα ασφαλές νήμα εκτέλεσης (Java.perform). Μέσω της συνάρτησης Java.use, το Frida εντοπίζει την ήδη φορτωμένη κλάση MainActivity στη μνήμη RAM. Η ουσία της δυναμικής παράκαμψης έγκειται στην ιδιότητα .implementation. Η αυθεντική λογική της συνάρτησης login() (η οποία περιέχει τους ελέγχους equals με τα διαπιστευτήρια) αντικαθίσταται πλήρως από τη νέα custom συνάρτηση. Η νέα συνάρτηση αγνοεί τα δεδομένα εισόδου του χρήστη και απλώς αξιοποιεί το Android SDK (μέσω των κλάσεων android.widget.Toast και java.lang.String) για να εμφανίσει απευθείας το μήνυμα επιτυχούς εισόδου.

Με το αρχείο bypass.js αποθηκευμένο στον φάκελο εργασίας, ανοίχτηκε ένα νέο παράθυρο τερματικού (διατηρώντας τον διακομιστή ενεργό στο προηγούμενο). Έχοντας ανοιχτή την εικονική συσκευή, δόθηκε η εντολή εκτέλεσης της επίθεσης: **frida -U -f com.unipi.project02 -l bypass.js**. Η παράμετρος «-U» όρισε τη σύνδεση με την εικονική συσκευή, η παράμετρος «-f» εξανάγκασε την εφαρμογή να εκκινήσει από το μηδέν, και η παράμετρος «-l» τροφοδότησε τον κώδικα παρέμβασης.



```
Command Prompt - frida -U
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Kalaia>cd Desktop\Android_Project_2

C:\Users\Kalaia\Desktop\Android_Project_2>frida -U -f com.unipi.project02 -l bypass.js

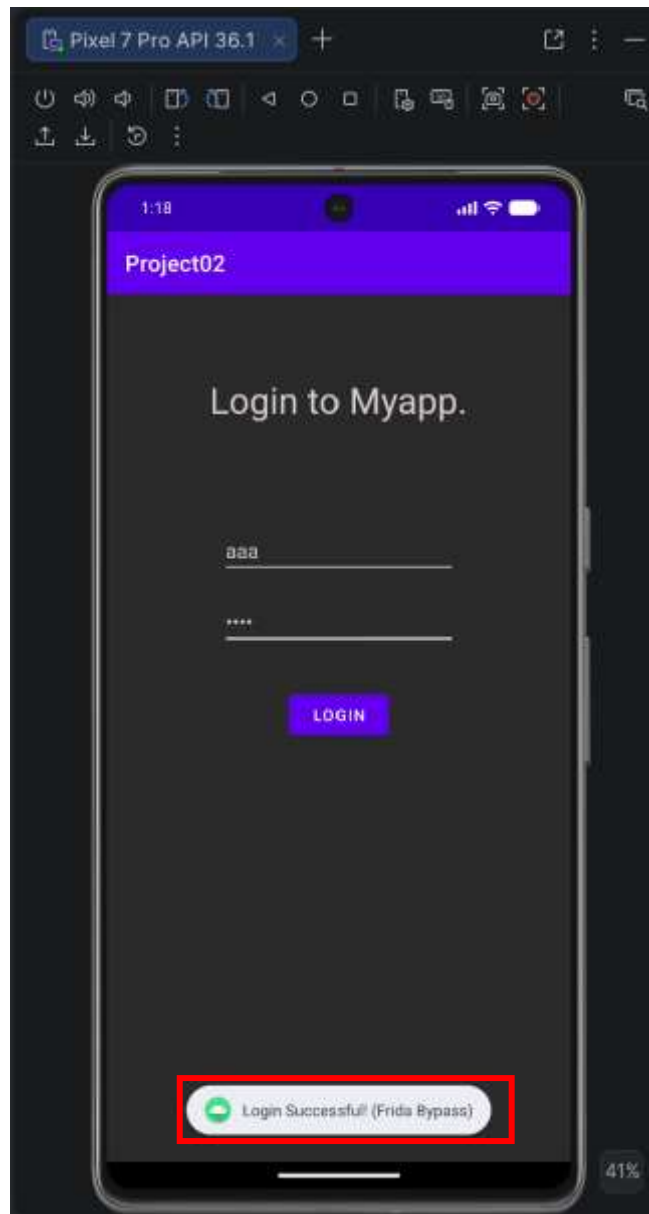
Frida 17.14.1 - A world-class dynamic instrumentation toolkit

Commands:
  help      -> Displays the help system
  object?   -> Display information about 'object'
  exit/quit -> Exit

More info at https://frida.re/docs/home/

Connected to Android Emulator 5554 (id=emulator-5554)
Spawned 'com.unipi.project02'. Resuming main thread!
[Android Emulator 5554]:com.unipi.project02 ]-> [*] To Frida Script φορτώθηκε επιτυχώς!
```

Μόλις η εφαρμογή εκκίνησε στην οθόνη του προσομοιωτή, πραγματοποιήθηκε δοκιμή εισάγοντας τυχαία, μη έγκυρα δεδομένα τόσο στο πεδίο Username (π.χ. «aaa») όσο και στο πεδίο Password (π.χ. «bbbb»). Με την ενεργοποίηση του πλήκτρου «LOGIN», ο κώδικας της παρέμβασης (hook) υπερίσχυσε άμεσα. Η εφαρμογή, έχοντας χάσει την αρχική της λογική επαλήθευσης, επέτρεψε την είσοδο προβάλλοντας το προσαρμοσμένο μήνυμα ειδοποίησης: «Login Successful! (Frida Bypass)».



Η συγκεκριμένη συμπεριφορά αποτελεί την αδιαμφισβήτητη επιβεβαίωση της επιτυχίας της μεθόδου 3, αποδεικνύοντας πρακτικά πώς η δυναμική ανάλυση μπορεί να υποτάξει τη ροή εκτέλεσης μιας εφαρμογής σε πραγματικό χρόνο, ακυρώνοντας εντελώς τους τοπικούς μηχανισμούς ασφαλείας της.

Συμπέρασμα

Η παρούσα άσκηση ανέδειξε την πολυπλοκότητα αλλά και τις πολλαπλές ευπάθειες των μηχανισμών αυθεντικοποίησης στο οικοσύστημα του Android. Η εφαρμογή επιχειρήθηκε να προστατευτεί μεταφέροντας τον ευαίσθητο κώδικα (Flag)

σε μια εξωτερική native βιβλιοθήκη C/C++ (JNI). Παρά το αυξημένο επίπεδο δυσκολίας σε σχέση με την απλή αποθήκευση της προηγούμενης άσκησης (UC1), αποδείχθηκε ότι καμία προστασία από μόνη της δεν είναι απαραβίαστη. Η προσέγγιση πραγματοποιήθηκε μέσα από τρεις διαφορετικούς πυλώνες του Reverse Engineering:

1. **Λογική παράκαμψη (Smali Patching):** Επιβεβαίωσε ότι η εμπιστοσύνη στη στατική ροή του κώδικα είναι επισφαλής, καθώς η εφαρμογή ανακατασκευάστηκε ώστε να δέχεται λανθασμένους κωδικούς.
2. **Αντίστροφη μηχανική native κώδικα (Ghidra):** Απέδειξε ότι ο κώδικας μηχανής, ακόμη και με τεχνικές απόκρυψης όπως τα stack strings, μπορεί να αποκρυπτογραφηθεί πλήρως αποκαλύπτοντας το πολυπόθητο Flag.
3. **Δυναμική ανάλυση (Frida):** Κατέδειξε τη δύναμη της παρέμβασης κατά τον χρόνο εκτέλεσης (runtime), όπου η λογική της εφαρμογής μπορεί να παραβιαστεί, χωρίς να πειραχτεί το αρχείο APK.

Συνολικά, η εργασία επιβεβαιώνει την αναγκαιότητα πολλαπλών επιπέδων ασφάλειας, όπως η χρήση server-side validation και ισχυρής κρυπτογράφησης, αντί της απλής απόκρυψης διαπιστευτηρίων στον κώδικα της συσκευής.

Βιβλιογραφία

- Releases · skylot/jadx. (n.d.). Retrieved from <https://github.com/skyloot/jadx/releases>
- Releases · patrickfav/uber-apk-signer. (n.d.). Retrieved from <https://github.com/patrickfav/uber-apk-signer/releases>
- Releases · NationalSecurityAgency/ghidra. (n.d.). Retrieved from <https://github.com/NationalSecurityAgency/ghidra/releases>
- Releases · frida/frida. (n.d.). Retrieved from <https://github.com/frida/frida/releases>
- OWASP MASTG - OWASP Mobile Application Security. (n.d.). Retrieved from <https://mas.owasp.org/MASTG/>
- ApkTool | ApkTool. (n.d.). Retrieved from <https://ibotpeaches.github.io/Apktool/>
- Frida • A world-class dynamic instrumentation toolkit. (n.d.). Retrieved from <https://frida.re/>